

WARZONE.GG

CODM Tournament Platform

South Africa · 2026

PROJECT DOCUMENTATION HANDOVER

Frontend MVP · Backend Architecture · Database Schema · Implementation Guide

Document Purpose	Complete reference for continuing development across sessions and with different AI assistants.
Tech Stack	HTML · Tailwind CSS · Vanilla JavaScript · Node.js · Express.js · Supabase · PostgreSQL
Frontend Status	MVP COMPLETE — 9 pages built and downloadable as warzongg-mvp.zip
Backend Status	PLANNED — architecture defined, ready to implement
Author	Mlungisi Nxumalo + Claude (Anthropic)
Version	1.0 — Initial Handover Document
Date	20 April 2026

Table of Contents

1. Project Overview

1.1 What is WARZONE.GG?

WARZONE.GG is a full-stack web application designed to serve as the official tournament management platform for Call of Duty: Mobile (CODM) players in South Africa. The platform enables players and teams to register for tournaments, track live brackets and match results, view leaderboards, and receive announcements — all through a single, centralised website.

The platform is built with a dark, aggressive military/tactical visual identity, using the Barlow Condensed and Rajdhani font families, a near-black colour scheme, and gold accent colours to reflect the competitive esports aesthetic.

1.2 Purpose

The primary purpose of WARZONE.GG is to solve a real problem in the South African CODM community: there is no dedicated, professional platform for organising and tracking local CODM tournaments. Current alternatives rely on WhatsApp groups, spreadsheets, and manual bracket management — all of which are error-prone and unprofessional.

WARZONE.GG aims to:

- Provide a single platform where teams can register, compete, and track standings.
- Give tournament organisers (admins) a professional dashboard to manage events, approve teams, and submit results.
- Build community through public leaderboards, team profiles, and match history.
- Distribute prize pools transparently, with structured breakdowns visible to all participants.

1.3 Target Users

User Type	Role	Primary Actions
Player / Fan	Public visitor	Browse tournaments, view leaderboard, read rules, watch bracket
Team Captain	Registered user	Register team, manage roster, view schedule, check results
Admin / Organiser	Privileged user	Create tournaments, approve teams, submit scores, publish announcements

1.4 Key Features Summary

Feature	Description	Status
Home Page	Hero, live stats, featured tournaments, recent results, announcements	Built (MVP)
Tournament Listings	Filtered grid of all tournaments with modal detail view	Built (MVP)
Live Bracket Viewer	Elimination bracket with real match scores and live indicators	Built (MVP)
Leaderboard	Sortable team standings with KD, points, earnings, podium	Built (MVP)
Match Schedule	Day-grouped schedule with status badges and notifications	Built (MVP)
Team Registration	4-step form: tournament, team info, roster, captain contact	Built (MVP)
Rules Page	7-section rulebook with jump navigation	Built (MVP)
Admin Dashboard	5-tab panel: overview, teams, matches, create tournament, announcements	Built (MVP)
Team Profile	Dynamic team page with roster, match history, stats	Built (MVP)
User Authentication	Email/password login, JWT sessions, role-based access	Backend required
Real Database	All data persisted in PostgreSQL via Supabase	Backend required
	Supabase Realtime pushes bracket updates to all	

Live Score Updates	viewers	Backend required
File Uploads	Team logos and match screenshots via Supabase Storage	Backend required
Email Notifications	Approval confirmations, prize notices via Resend/Mailgun	Backend required

2. Frontend MVP

2.1 Tech Stack

Technology	Purpose	How it is used
HTML5	Page structure	9 separate .html files, one per page/route
Tailwind CSS	Utility-first styling	Loaded via CDN (cdn.tailwindcss.com) — no build step required
Vanilla JavaScript	Interactivity & data	Inline <script> tags per page; no framework
Google Fonts	Typography	Barlow Condensed (headings), Rajdhani (body), Share Tech Mono (labels)
localStorage	Temporary persistence	Stores new team registrations across pages in MVP only

2.2 File Structure

The MVP is delivered as a ZIP archive (warzongg-mvp.zip). Unzip and open index.html directly in a browser — no server required for the frontend.

File	Page	Notes
index.html	Home	Root file — open this first
pages/tournaments.html	Tournament listings	Filter by mode and status; modal detail view
pages/bracket.html	Live bracket	Operation Warfront bracket with result filtering
pages/leaderboard.html	Season leaderboard	Sortable table; podium for top 3
pages/schedule.html	Match schedule	Day-grouped; filter by tournament
pages/register.html	Team registration	4-step form; saves to localStorage in MVP
pages/rules.html	Rules & regulations	7 sections with anchor jump links
pages/admin.html	Admin dashboard	Login gate (admin/admin123); 5-tab management panel
pages/team-profile.html	Team profile	Dynamic page — loaded via ?id= URL parameter
js/data.js	Mock data store	WZ object containing all tournaments, teams, matches
js/main.js	Shared utilities	Toast, modals, nav highlight, formatters
js/nav.js	Shared navigation	injectNav() called by every page in /pages/
css/style.css	Custom styles	Fonts, custom classes, animations on top of Tailwind

2.3 Shared JavaScript Utilities

Every page in the /pages/ directory calls injectNav(pageName) from nav.js to render the navigation bar. The following utility functions are available globally via main.js:

Function	Purpose
showToast(msg, type)	Displays a slide-in notification (success/error/info)
openModal(id) / closeModal(id)	Shows or hides a modal overlay by element ID
toggleMobileMenu()	Toggles the mobile hamburger navigation
statusBadge(status)	Returns HTML string for a coloured status pill
slotsBar(registered, slots)	Returns HTML for a tournament slot progress bar
fmtPrize(n)	Formats a number as South African Rand (e.g. R 50,000)
rankBadge(n)	Returns HTML for a gold/silver/bronze rank badge

IMPORTANT FOR NEXT SESSION: When connecting the frontend to the backend, the primary change is replacing WZ.tournaments, WZ.teams etc. in data.js with fetch() calls to your Express API endpoints. The HTML, CSS, and nav structure do not change.

3. Backend Architecture

3.1 Chosen Tech Stack

The backend stack was chosen to stay as close to the MVP's JavaScript-only approach as possible, while adding the robustness of a real database and authentication system.

Layer	Technology	Why chosen
Runtime	Node.js 20+	Same language as frontend (JavaScript) — no context switch
Web Framework	Express.js	Minimal, unopinionated; easy to add routes exactly where needed
Database	PostgreSQL (via Supabase)	Relational DB — ideal for tournaments, teams, matches relationships
BaaS	Supabase	Provides DB, Auth, Realtime, and Storage in one free-tier platform
Authentication	Supabase Auth + JWT	Handles password hashing, token generation, and refresh tokens
Email	Resend or Mailgun	Transactional emails for approvals, prize confirmations
Frontend Host	Netlify	Free static hosting; deploys automatically from GitHub
API Host	Render	Free Node.js hosting; auto-deploys from GitHub; upgrade for tournaments
Source Control	GitHub	Connects Netlify and Render for auto-deployment on push

3.2 System Architecture

The production system has four distinct layers that communicate in sequence:

1. User opens warzone.gg in browser (hosted on Netlify CDN).
2. HTML page loads. JavaScript makes `fetch()` calls to the Express API on Render.

3. Express API validates the request, checks JWT if the route is protected, then queries Supabase.
4. Supabase returns data from PostgreSQL. The API formats it as JSON and sends it back.
5. The browser renders the response into the existing HTML templates.

3.3 Express API Folder Structure

Recommended project layout for the backend repository:

```
warzongg-api/

├─ server.js           ─ App entry point; starts Express server

├─ .env               ─ Environment variables (NEVER commit to GitHub)

├─ .env.example       ─ Safe template for other devs

├─ package.json

├─ config/

|   └─ supabase.js    ─ Initialises Supabase client

├─ middleware/

|   ├─ auth.js        ─ JWT verification middleware
|   ├─ adminOnly.js   ─ Checks user has admin role
|   └─ validate.js    ─ express-validator input rules

├─ routes/

|   ├─ auth.js        ─ POST /auth/register, /auth/login, /auth/logout
|   ├─ tournaments.js ─ GET/POST /tournaments, GET /tournaments/:id
|   ├─ teams.js       ─ POST /teams/register, PATCH /teams/:id/status
|   ├─ matches.js     ─ GET /matches, POST /matches (admin only)
|   ├─ leaderboard.js ─ GET /leaderboard
|   └─ admin.js       ─ GET /admin/dashboard (admin only)

└─ controllers/

    ├─ tournamentsController.js

    └─ teamsController.js
```

└─ matchesController.js

└─ authController.js

3.4 Environment Variables

Create a `.env` file in the project root. Never commit this file to GitHub. Add `.env` to your `.gitignore` immediately.

Variable	Value Source	Used in
SUPABASE_URL	Supabase project Settings > API	config/supabase.js
SUPABASE_ANON_KEY	Supabase project Settings > API	config/supabase.js
SUPABASE_SERVICE_KEY	Supabase project Settings > API (secret)	Admin operations only
JWT_SECRET	Generate randomly (e.g. <code>openssl rand -hex 32</code>)	middleware/auth.js
PORT	3000 (local) or set by Render automatically	server.js
FRONTEND_URL	https://warzonezagg.netlify.app (your Netlify URL)	CORS config in server.js
RESEND_API_KEY	resend.com dashboard	Email notifications

SECURITY NOTE: The `SUPABASE_SERVICE_KEY` bypasses all Supabase row-level security policies. Only use it in server-side code (Express), never in frontend JavaScript that runs in the browser.

4. Database Schema (PostgreSQL / Supabase)

4.1 Overview

The database uses 7 core tables. All tables use UUID primary keys (Supabase default). Timestamps use `timestampz` (timestamp with timezone). All foreign keys have `ON DELETE CASCADE` unless otherwise noted.

Table	Purpose	Key relationships
users	Authentication accounts and roles	Referenced by teams (captain)
teams	Registered competing teams	Belongs to users (captain); has many players
players	Individual roster members	Belongs to teams
tournaments	Tournament events	Has many registrations, matches
registrations	Team applications to tournaments	Belongs to teams and tournaments
matches	Individual match results	Belongs to tournaments; references teams
announcements	Admin-published site notices	Standalone; created by admin user

4.2 Table: users

Managed primarily by Supabase Auth. A corresponding profile row is auto-created in this table on signup via a Supabase database trigger.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Auto-generated by Supabase Auth
email	text	UNIQUE NOT NULL	Login email address
username	text	UNIQUE NOT NULL	Public display name
role	text	DEFAULT 'player'	Values: 'player' or 'admin'
whatsapp	text	NULLABLE	Contact number
discord	text	NULLABLE	Discord tag
created_at	timestampz	DEFAULT now()	Account creation time
updated_at	timestampz	DEFAULT now()	Last profile update

4.3 Table: teams

Represents a competing team registered on the platform. One captain (user) owns a team. A team can be registered for multiple tournaments via the registrations table.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()	Unique team ID
name	text	UNIQUE NOT NULL	Full team name (e.g. GhostSquad ZA)
tag	text	NOT NULL	Short team tag max 4 chars (e.g. GSZ)
region	text	NOT NULL	Home province (e.g. Gauteng)
captain_id	uuid	REFERENCES users(id)	The user who registered the team
logo_url	text	NULLABLE	Supabase Storage URL for team logo
wins	integer	DEFAULT 0	Total wins across all tournaments
losses	integer	DEFAULT 0	Total losses across all tournaments
points	integer	DEFAULT 0	Total season points
earnings	integer	DEFAULT 0	Total prize money earned (ZAR cents)
created_at	timestampz	DEFAULT now()	Team registration date

4.4 Table: players

Individual players belonging to a team roster. Each team can have up to 6 players (5 starters + 1 substitute).

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()	Unique player row ID
team_id	uuid	REFERENCES teams(id) ON DELETE CASCADE	Team this player belongs to
user_id	uuid	REFERENCES users(id) NULLABLE	Links to user account if registered
ign	text	NOT NULL	In-game name (IGN)
codm_uid	text	NOT NULL	12-digit CODM UID for verification
role	text	DEFAULT 'player'	Values: 'igl', 'fragger', 'sniper', 'support', 'rusher', 'substitute'
device	text	DEFAULT 'android'	Device type: android, ios, emulator
is_substitute	boolean	DEFAULT false	Whether this is the sub slot
created_at	timestamptz	DEFAULT now()	Date added to roster

4.5 Table: tournaments ← MOST IMPORTANT TABLE

The central table of the application. Every tournament event is a row here. All other tables (registrations, matches) reference this table.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()	Unique tournament ID
name	text	NOT NULL	Full event name (e.g. Operation Warfront — Grand Finals)
tag	text	NOT NULL	Short identifier (e.g. OWF)
mode	text	NOT NULL	Game mode label (e.g. Ranked 5v5, Squad BR)
type	text	NOT NULL	Internal type key: ranked, br, duos, solo
status	text	DEFAULT 'upcoming'	Values: upcoming, open, live, completed — controls what buttons show
format	text	NOT NULL	Format description (e.g. Hardpoint + S&D)
region	text	DEFAULT 'ZA'	Target region
start_date	date	NOT NULL	Tournament start date
reg_deadline	date	NOT NULL	Registration closes date
max_slots	integer	NOT NULL	Maximum number of teams allowed

registered_count	integer	DEFAULT 0	Current number of approved teams (auto-updated by trigger)
prize_total	integer	NOT NULL	Total prize pool in ZAR cents
entry_fee	integer	DEFAULT 0	Entry fee per team in ZAR cents
prize_1st	integer	DEFAULT 0	1st place prize in ZAR cents
prize_2nd	integer	DEFAULT 0	2nd place prize in ZAR cents
prize_3rd	integer	DEFAULT 0	3rd place prize in ZAR cents
prize_mvp	integer	DEFAULT 0	MVP award in ZAR cents
description	text	NULLABLE	Long-form tournament description
winner_team_id	uuid	REFERENCES teams(id) NULLABLE	Populated when status = completed
created_by	uuid	REFERENCES users(id)	Admin who created this tournament
created_at	timestampz	DEFAULT now()	When the tournament was created
updated_at	timestampz	DEFAULT now()	Last modification timestamp

*Prize values are stored in cents (integers) to avoid floating-point errors. Always divide by 100 to display to users.
Example: `prize_total = 5000000` displays as R 50,000.*

4.6 Table: registrations

A registration is a team's application to join a specific tournament. One team can be registered in multiple tournaments (one row per team per tournament). Approval happens here.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()	Unique registration ID
tournament_id	uuid	REFERENCES tournaments(id) ON DELETE CASCADE	Which tournament
team_id	uuid	REFERENCES teams(id) ON DELETE CASCADE	Which team is applying
status	text	DEFAULT 'pending'	Values: pending, approved, rejected — admin changes this
game_mode	text	NOT NULL	Selected game mode for this registration
device_type	text	DEFAULT 'android'	Captain's primary device
notes	text	NULLABLE	Admin notes on the registration
submitted_at	timestampz	DEFAULT now()	When registration was submitted
reviewed_at	timestampz	NULLABLE	When admin approved or rejected
reviewed_by	uuid	REFERENCES users(id) NULLABLE	Admin who reviewed

--	--	--	--

UNIQUE constraint: (tournament_id, team_id) — a team can only register once per tournament. Add this in Supabase:
ALTER TABLE registrations ADD CONSTRAINT unique_reg UNIQUE (tournament_id, team_id);

4.7 Table: matches

Each row represents one played map/match within a tournament. A Bo3 series generates up to 3 rows. The bracket is constructed by querying this table grouped by round.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()	Unique match ID
tournament_id	uuid	REFERENCES tournaments(id) ON DELETE CASCADE	Parent tournament
round	text	NOT NULL	Values: group_stage, quarter_final, semi_final, grand_final
team_a_id	uuid	REFERENCES teams(id)	First competing team
team_b_id	uuid	REFERENCES teams(id) NULLABLE	Second team (null = TBD)
score_a	integer	NULLABLE	Team A score (null = not yet played)
score_b	integer	NULLABLE	Team B score (null = not yet played)
winner_id	uuid	REFERENCES teams(id) NULLABLE	Auto-set when scores are submitted
map_name	text	NULLABLE	e.g. Standoff, Rust, Nuketown
game_mode	text	NULLABLE	e.g. Hardpoint, S&D, Domination

status	text	DEFAULT 'scheduled'	Values: scheduled, live, completed
scheduled_at	timestampz	NULLABLE	Planned match start time
played_at	timestampz	NULLABLE	Actual match completion time
submitted_by	uuid	REFERENCES users(id) NULLABLE	Admin who submitted the result

4.8 Table: announcements

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()	Unique announcement ID
title	text	NOT NULL	Announcement headline
body	text	NOT NULL	Full announcement text
type	text	DEFAULT 'info'	Values: info, live, update, warning
tournament_id	uuid	REFERENCES tournaments(id) NULLABLE	If targeted at a specific event
published_by	uuid	REFERENCES users(id)	Admin author
created_at	timestampz	DEFAULT now()	Publication time

4.9 Entity Relationship Summary

The relationships between all tables are described below. Read (||) as 'exactly one' and (o{) as 'zero or many'.

Relationship	Cardinality	Description
users → teams	1 to many	One user (captain) can own many teams
teams → players	1 to many	One team has many players (max 6)
tournaments → registrations	1 to many	One tournament has many team registrations
teams → registrations	1 to many	One team can register for many tournaments
tournaments → matches	1 to many	One tournament contains many matches
teams → matches (as team_a)	1 to many	One team plays in many matches as team A
teams → matches (as team_b)	1 to many	One team plays in many matches as team B
users → announcements	1 to many	One admin user publishes many announcements
tournaments → announcements	0/1 to many	Announcements can optionally target one tournament
teams → tournaments (via winner_team_id)	0/1 to 1	At most one team wins a given tournament

5. API Endpoints Reference

5.1 Authentication Routes (/api/auth)

Method	Endpoint	Auth required	Description
POST	/api/auth/register	None	Create new user account (email, password, username)
POST	/api/auth/login	None	Returns JWT access token on success
POST	/api/auth/logout	JWT	Invalidates current session token
GET	/api/auth/me	JWT	Returns current user profile and role

5.2 Tournament Routes (/api/tournaments)

Method	Endpoint	Auth required	Description
GET	/api/tournaments	None	List all tournaments — supports ?status= and ?type= filters
GET	/api/tournaments/:id	None	Single tournament detail with prize breakdown
POST	/api/tournaments	Admin JWT	Create new tournament
PATCH	/api/tournaments/:id	Admin JWT	Update tournament fields (status, dates, prizes)
DELETE	/api/tournaments/:id	Admin JWT	Remove tournament and cascade-delete related data

5.3 Team & Registration Routes

Method	Endpoint	Auth required	Description
POST	/api/teams	JWT	Create a team (captain becomes owner)
GET	/api/teams/:id	None	Team profile with roster and match history
PATCH	/api/teams/:id	JWT (owner)	Update team info (name, tag, logo)
POST	/api/registrations	JWT	Register a team for a tournament
GET	/api/registrations	Admin JWT	List all registrations — supports ?status= filter
PATCH	/api/registrations/:id/status	Admin JWT	Approve or reject a team registration

5.4 Match & Leaderboard Routes

Method	Endpoint	Auth required	Description
GET	/api/matches	None	List matches — supports ?tournament_id= and ?round= filters
GET	/api/matches/:id	None	Single match detail
POST	/api/matches	Admin JWT	Submit a new match result (scores, map, mode, round)
PATCH	/api/matches/:id	Admin JWT	Correct a previously submitted result
GET	/api/leaderboard	None	Ranked teams by points — supports ?season= filter
GET	/api/schedule	None	Upcoming and recent matches grouped by date

5.5 Admin Routes (/api/admin)

Method	Endpoint	Auth required	Description
GET	/api/admin/dashboard	Admin JWT	Overview stats: pending count, active tournaments, recent activity
GET	/api/admin/teams	Admin JWT	All teams with status filter and search
POST	/api/admin/announcements	Admin JWT	Publish a new announcement
GET	/api/admin/announcements	Admin JWT	List all announcements including unpublished

5.6 Example: Register a Team (POST /api/teams)

Request body (JSON):

```
{  
  
  "name": "GhostSquad ZA",  
  
  "tag": "GSZ",  
  
  "region": "Gauteng",  
  
  "tournament_id": "uuid-of-tournament",  
  
  "game_mode": "Battle Royale",  
  
  "players": [  
  
    { "ign": "GhostX", "codm_uid": "123456789012", "role": "igl" },  
  
    { "ign": "Specter", "codm_uid": "234567890123", "role": "fragger" }  
  
  ],  
  
  "captain_name": "John Doe",  
  
  "whatsapp": "+27821234567",  
  
  "email": "team@example.com"  
  
}
```

Success response (201 Created):

```
{ "success": true, "team_id": "uuid", "registration_id": "uuid", "status": "pending" }
```

6. Authentication & Security

6.1 How Authentication Works

Supabase Auth handles the heavy lifting. The flow is:

1. User submits email + password to POST /api/auth/login.
2. Express calls `supabase.auth.signInWithPassword()` with those credentials.
3. Supabase validates and returns a JWT access token (expires in 1 hour).
4. Express sends the JWT back to the browser in the response body.
5. The frontend stores the JWT in memory (not `localStorage` — security risk) and attaches it as `Authorization: Bearer <token>` on every subsequent API request.
6. Protected Express routes run the auth middleware, which calls `supabase.auth.getUser(token)` to verify the token.
7. Admin routes additionally check that `user.role === 'admin'` in the database.

6.2 JWT Middleware (middleware/auth.js)

Paste this code into `middleware/auth.js` in your backend project:

```
const { createClient } = require('@supabase/supabase-js');

const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_ANON_KEY);

async function requireAuth(req, res, next) {

  const token = req.headers.authorization?.split('Bearer ')[1];

  if (!token) return res.status(401).json({ error: 'No token provided' });

  const { data: { user }, error } = await supabase.auth.getUser(token);

  if (error || !user) return res.status(401).json({ error: 'Invalid token' });

  req.user = user;

  next();

}

async function requireAdmin(req, res, next) {
```

```
await requireAuth(req, res, async () => {

  const { data } = await supabase.from('users').select('role').eq('id', req.user.id).single();

  if (data?.role !== 'admin') return res.status(403).json({ error: 'Admin only' });

  next();

});

}

module.exports = { requireAuth, requireAdmin };
```

6.3 Security Checklist

Before going live with a real tournament, verify all of the following:

Item	Status	How to fix if failing
.env is in .gitignore	Must pass	Run: echo '.env' >> .gitignore before first commit
CORS origin is not wildcard *	Must pass	Set origin: process.env.FRONTEND_URL in Express CORS config
Admin routes use requireAdmin middleware	Must pass	Add requireAdmin to every /api/admin route
Passwords never stored in plain text	Auto (Supabase handles)	Supabase Auth uses bcrypt internally
SQL injection not possible	Auto (Supabase SDK uses parameterised queries)	Never use raw SQL string concatenation
SUPABASE_SERVICE_KEY only used server-side	Must pass	Never import it in frontend JS files
Input validation on all POST routes	Must implement	Use express-validator on every route that accepts body data
Rate limiting on auth routes	Recommended	npm install express-rate-limit; add to /api/auth routes
HTTPS enforced	Auto (Netlify + Render both enforce)	Do not use plain HTTP in production links

7. Supabase Setup — Step by Step

7.1 Creating the Project

1. Go to supabase.com and sign in (free with GitHub).
2. Click New Project. Name it warzongg. Choose a strong database password and save it safely.
3. Select the Africa (Cape Town) region if available, otherwise choose the closest option.
4. Wait ~2 minutes for the project to provision.
5. Go to Project Settings > API. Copy the Project URL and anon/public key into your .env file.

7.2 Running the SQL Schema

In your Supabase project, go to the SQL Editor and run each CREATE TABLE statement. The order matters due to foreign key dependencies:

1. users table first (no foreign key dependencies)
2. teams (depends on users)
3. players (depends on teams and users)
4. tournaments (depends on users)
5. registrations (depends on teams and tournaments)
6. matches (depends on tournaments and teams)
7. announcements (depends on users and tournaments)

7.3 Row Level Security (RLS)

Supabase enables Row Level Security (RLS) on all tables by default. This means no data is accessible unless you explicitly create policies. Add these essential policies:

Table	Policy	Rule
tournaments	Public read	Allow SELECT for all (including unauthenticated users)
tournaments	Admin write	Allow INSERT, UPDATE, DELETE only where user role = admin
teams	Public read	Allow SELECT for all
teams	Owner write	Allow UPDATE only where captain_id = auth.uid()
registrations	Owner read	Allow SELECT only where team captain_id = auth.uid() OR user role = admin
registrations	Authenticated insert	Allow INSERT for logged-in users
matches	Public read	Allow SELECT for all
matches	Admin write	Allow INSERT, UPDATE only where user role = admin
announcements	Public read	Allow SELECT for all
announcements	Admin write	Allow INSERT only where user role = admin

7.4 Auto-create User Profile Trigger

When a user signs up through Supabase Auth, a corresponding row must be created in the public users table. Add this trigger in the SQL Editor:

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
```

```

RETURNS trigger AS $$

BEGIN

    INSERT INTO public.users (id, email, username, role)

VALUES (NEW.id, NEW.email, NEW.raw_user_meta_data->>'username', 'player');

RETURN NEW;

END;

$$ LANGUAGE plpgsql SECURITY DEFINER;


CREATE TRIGGER on_auth_user_created

AFTER INSERT ON auth.users

FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

```

7.5 Enabling Realtime

To enable live bracket updates, go to your Supabase project > Database > Replication. Enable Realtime for the matches table. Then in your frontend JavaScript:

```

const channel = supabase.channel('match-updates')

.on('postgres_changes', {

    event: 'UPDATE', schema: 'public', table: 'matches'

}, (payload) => {

    updateBracketDisplay(payload.new);

})

.subscribe();

```

8. Deployment Guide

8.1 GitHub Setup

Both Netlify and Render connect to GitHub repositories for automatic deployment. Create two repositories:

Repository	Contents	Connects to
warzongg-frontend	All HTML, CSS, JS files from the MVP ZIP	Netlify
warzongg-api	The Express.js backend folder	Render

After every code change, run `git push origin main`. Both Netlify and Render detect the push and redeploy automatically within 1-2 minutes.

8.2 Deploying the Frontend to Netlify

1. Go to netlify.com and sign in with GitHub.
2. Click Add new site > Import an existing project > Connect to GitHub.
3. Select your warzongg-frontend repository.
4. Build command: leave blank (no build step — pure HTML/CSS/JS).
5. Publish directory: / (root of the repository).
6. Click Deploy site. Netlify gives you a URL like `warzongg.netlify.app`.
7. In Site settings > Domain, add a custom domain if you have one (e.g. `warzone.gg`).

8.3 Deploying the API to Render

1. Go to render.com and sign in with GitHub.
2. Click New > Web Service. Connect your warzongg-api repository.
3. Environment: Node. Build command: `npm install`. Start command: `node server.js`.
4. Add all environment variables from your `.env` file under Environment > Add Environment Variable.
5. Click Create Web Service. Render gives you a URL like `warzongg-api.onrender.com`.

IMPORTANT: The free tier on Render spins down after 15 minutes of inactivity and takes ~30 seconds to wake up on the next request. For a live tournament day, upgrade to the paid Starter plan (\$7/month) to keep the service always-on.

8.4 Connecting Frontend to Backend

After both are deployed, update one line in your frontend JavaScript. In `js/data.js` (or a new `js/config.js`), set:

```
const API_URL = 'https://warzongg-api.onrender.com';
```

Then replace every mock data reference with a `fetch()` call. Example — loading tournaments:

```
// BEFORE (MVP mock data):

const tournaments = WZ.tournaments;

// AFTER (real backend):

const res = await fetch(`${API_URL}/api/tournaments`);

const { data: tournaments } = await res.json();
```

8.5 CORS Configuration in Express

Add this to `server.js` before any route definitions:

```
const cors = require('cors');

app.use(cors({

  origin: process.env.FRONTEND_URL, // e.g. https://warzongg.netlify.app

  methods: ['GET', 'POST', 'PATCH', 'DELETE'],

  allowedHeaders: ['Content-Type', 'Authorization']

}));
```

9. Key Data Flows

9.1 Team Registration Flow

This is the most important user flow. Follow each step carefully when implementing.

Step	Actor	Action	Result
1	Captain (browser)	Fills out registration form and submits	POST /api/registrations sent to Express
2	Express	requireAuth middleware checks JWT token	Request rejected if not logged in
3	Express	Validates all form fields with express-validator	Returns 400 error with field details if invalid
4	Express	Checks tournament has open slots (registered_count < max_slots)	Returns 409 error if tournament is full
5	Supabase	Creates teams row + players rows in transaction	New team exists in DB with pending registration
6	Supabase	Creates registrations row with status = pending	Admin can now see it in dashboard
7	Express	Returns { success: true, registration_id, status: pending }	Frontend shows success message
8	Admin (browser)	Reviews in Admin > Teams tab	PATCH /api/registrations/:id/status body: {status: approved}
9	Supabase	Updates registration status; increments registered_count on tournament	Team is now officially in the tournament
10	Resend/Mailgun	Email sent to captain confirming approval	Captain receives entry fee payment instructions

9.2 Match Result Submission Flow

Step	Actor	Action	Result
1	Admin	Submits scores in Admin > Matches tab	POST /api/matches with scores, map, mode, round
2	Express	requireAdmin middleware — blocks non-admins	Only verified admin can submit results
3	Express	Determines winner_id from score_a vs score_b	winner_id set automatically
4	Supabase	Creates matches row with status = completed	Result stored permanently
5	Supabase Realtime	Broadcasts UPDATE event to all subscribed clients	All browsers watching the bracket page update instantly
6	Frontend	channel listener calls updateBracketDisplay()	Bracket re-renders with new scores, no page refresh needed
7	Express trigger	Updates team wins/losses/points in teams table	Leaderboard reflects new standings

10. Beginner Reference Glossary

Definitions of every technical term used in this document, written for someone with minimal backend experience.

Term	Plain English Explanation
API (Application Programming Interface)	A set of URLs your frontend can call to get or send data. Like a waiter — you give your order (request), the kitchen (server) prepares it, the waiter brings it back (response).
REST API	A style of API where each URL represents a resource (e.g. /tournaments) and HTTP verbs (GET, POST, PATCH, DELETE) describe what you want to do with it.
Express.js	A JavaScript library that makes it easy to build APIs with Node.js. You define routes like <code>app.get('/tournaments', ...)</code> and Express handles the rest.
Node.js	A programme that lets JavaScript run on a server (outside the browser). Your backend code is JavaScript, just like your frontend.
Supabase	A service that gives you a database, authentication, and file storage — all managed for you. You do not need to install PostgreSQL yourself.
PostgreSQL	The actual database engine that stores your data in tables. Supabase runs it for you.
JWT (JSON Web Token)	A signed string that proves who you are. After login, the server gives you a JWT. You include it in every future request as proof of identity.
Middleware	A function in Express that runs before your route handler. Used for checking tokens, validating inputs, or logging requests.
Row Level Security (RLS)	A Supabase feature that controls who can read or write which rows in a table. Like a bouncer for your data.
CORS (Cross-Origin Resource Sharing)	A browser security rule that blocks your frontend from calling an API on a different domain unless the API explicitly allows it. You configure this in Express.

Environment Variable	A value stored outside your code (in a .env file or on the hosting platform). Used for secrets like database passwords that must not be visible in GitHub.
UUID (Universally Unique Identifier)	A randomly generated ID like 8f14e45f-ceed-467a-a866-051f3c10e9f3. Used as primary keys instead of sequential integers for security.
Foreign Key	A column in one table that references a row in another table. e.g. team_id in players refers to the id in teams. Enforces data integrity.
ON DELETE CASCADE	When a parent row is deleted, all child rows are automatically deleted too. e.g. deleting a tournament removes all its matches and registrations.
Realtime (Supabase)	Supabase can push database changes to all connected browsers instantly using WebSockets. Used for live bracket updates.
CDN (Content Delivery Network)	A network of servers distributed globally. Netlify uses a CDN so your HTML loads quickly for users anywhere in the world.
Deployment	The process of putting your code on a server so real users can access it on the internet, as opposed to running it locally on your laptop.
GitHub	A website that stores your code online (version control). Both Netlify and Render watch your GitHub repo and auto-redeploy when you push changes.

11. Recommended Next Steps

Phase 1 — Project Setup (Day 1)

1. Install Node.js (nodejs.org) on your computer.
2. Install VS Code (code.visualstudio.com) as your code editor.
3. Install Git (git-scm.com) for version control.
4. Create a GitHub account if you do not have one.
5. Create two GitHub repositories: warzongg-frontend and warzongg-api.
6. Extract the warzongg-mvp.zip and push those files to warzongg-frontend.

Phase 2 — Supabase Setup (Day 1–2)

1. Create a Supabase account at supabase.com.
2. Create a new project called warzongg in the Africa region.
3. Run all 7 CREATE TABLE statements from Section 4 in the SQL Editor.
4. Set up Row Level Security policies from Section 7.3.
5. Add the user profile trigger from Section 7.4.
6. Enable Realtime for the matches table.
7. Copy your SUPABASE_URL and SUPABASE_ANON_KEY.

Phase 3 — Express API (Days 2–5)

1. Run npm init in your warzongg-api folder.
2. Install packages: npm install express cors dotenv @supabase/supabase-js express-validator
3. Create the folder structure from Section 3.3.
4. Create .env with all variables from Section 3.4.
5. Build the auth routes first — test login and registration before anything else.
6. Build tournament routes — GET all, GET by id, POST new (admin only).
7. Build team registration route — POST /api/registrations.
8. Build admin approval route — PATCH /api/registrations/:id/status.
9. Build match submission and leaderboard routes.
10. Test every route with Postman or Insomnia before connecting the frontend.

Phase 4 — Connect Frontend (Days 5–7)

1. Push warzongg-api to GitHub and deploy to Render.
2. Add the Render API URL to your .env as FRONTEND_URL and configure CORS.
3. In warzongg-frontend, replace WZ.tournaments with fetch() calls page by page.
4. Update the admin.html login to call POST /api/auth/login and store the JWT.
5. Update register.html to call POST /api/registrations.
6. Test the full registration-to-approval flow end-to-end.

7. Add Supabase Realtime listener to bracket.html.
8. Deploy updated frontend to Netlify.

Phase 5 — Pre-Launch (Days 7–10)

1. Run through the full security checklist from Section 6.3.
2. Register a domain name (warzone.co.za or similar) and point it to Netlify.
3. Upgrade Render to the Starter plan (\$7/month) before the first live tournament.
4. Create the first real admin account in Supabase Auth and set role = admin.
5. Create the first tournament in the admin dashboard.
6. Announce registration opening to the community.

When continuing this project with a new AI assistant or in a new session, share this document. It contains everything needed to understand the project from scratch: purpose, file structure, all database tables with column definitions, API endpoints, security approach, and deployment instructions.

12. Quick Reference Card

12.1 Key URLs

Fill these in as your project progresses:

Frontend (Netlify)	_____ .netlify.app
API (Render)	_____ .onrender.com
Supabase Dashboard	https://supabase.com/dashboard/project/_____
GitHub — Frontend	https://github.com/_____/warzongg-frontend
GitHub — API	https://github.com/_____/warzongg-api
Custom Domain	warzone.co.za or _____

12.2 Important Passwords & Keys (fill in securely)

DO NOT store actual keys in this document if you share it. Use a password manager like Bitwarden (free) instead.

Supabase DB Password	Stored in: _____
SUPABASE_URL	Found in: Supabase > Project Settings > API
SUPABASE_ANON_KEY	Found in: Supabase > Project Settings > API
SUPABASE_SERVICE_KEY	Found in: Supabase > Project Settings > API (secret key)
JWT_SECRET	Generated by you — run: openssl rand -hex 32
Admin account email	_____

12.3 Installed NPM Packages

Package	Version	Purpose
express	^4.18.0	Web framework — handles HTTP routes
cors	^2.8.5	Allows frontend to call the API
dotenv	^16.0.0	Loads .env variables into process.env
@supabase/supabase-js	^2.x.x	Official Supabase client library
express-validator	^7.x.x	Input validation middleware
express-rate-limit	^7.x.x	Rate limiting for auth routes
nodemon	^3.x.x (dev only)	Auto-restarts server on file changes during development

12.4 Admin Dashboard Credentials (MVP demo)

Username	admin
Password	admin123
Note	This is the demo credential for the MVP only. Replace with Supabase Auth in production.

